MODULE 11 · BLOCK 3 - DATA COLLECTIONS

Lists I: indexing, slicing and methods

Positions from the front and the back, slices with steps, and every list method the exam uses.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 11 of 28

admin@sthithapragna.com

A list is an ordered sequence you can change. Positions start at 0, negative positions count back from the end, and a slice stops before its end.

Square brackets, commas, any types

```
1  empty = []
2  mixed = [1, "two", 3.0, True, None]
3  nested = [[1, 2], [3]]
4  letters = list("hi")
5  print(len(mixed), len(nested), letters)   # 5 2 ['h', 'i']
```

2 A list can hold any mixture of types, None included.

3 A list inside a list counts as one element. len(nested) is 2.

4 list() turns any sequence into a list: a string becomes its characters.

Every element has a positive and a negative index

ELEMENT	"A"	"B"	"C"	"D"	"E"
Positive index	0	1	2	3	4
Negative index	-5	-4	-3	-2	-1

THE RULE

The last element is always [-1], and the first is [0]. For a list of length n, the valid indexes are -n to n - 1.

Reading and changing single elements

```
1  data = [10, 20, 30, 40]
2  print(data[0], data[-1], data[len(data) - 1])   # 10 40 40
3  data[1] = 25                                     # lists can change
4  print(data)                                      # [10, 25, 30, 40]
5  print(data[data[0] // 10])                       # data[1], so 25
```

3 Assigning to an index replaces that element in place.

5 The index can be any expression. Work out the inner one first.

What goes wrong, and what does not

EXPRESSION, WITH DATA = [1, 2, 3]	RESULT
`data[3]`	IndexError: list index out of range
`data[-4]`	IndexError: list index out of range
`data[1.0]`	TypeError: list indices must be integers or slices
`data[int(-1 / 2)]`	1: int(-0.5) is 0, so this is data[0]

ON THE EXAM

Positions outside -n to n - 1 raise IndexError. An index must be an int, even 1.0 is refused.

list[start:stop] gives a new list

```
1  a = [1, 2, 3, 4, 5]
2  print(a[1:3])      # [2, 3]        start included, stop excluded
3  print(a[:2])       # [1, 2]        from the beginning
4  print(a[2:])       # [3, 4, 5]     to the end
5  print(a[-2:])      # [4, 5]        the last two
6  print(a[:])        # [1, 2, 3, 4, 5]  a full copy
7  print(a[1:100])    # [2, 3, 4, 5]  no error
```

2 Exactly like range: from index 1 up to, but not including, index 3.

5 a[-n:] is the last n elements, whatever the list's length.

7 Slices never raise IndexError. Out-of-range ends are cut back to fit.

list[start:stop:step]

```
1  a = [1, 2, 3, 4, 5]
2  print(a[::2])      # [1, 3, 5]        every second element
3  print(a[1::2])     # [2, 4]
4  print(a[::-1])     # [5, 4, 3, 2, 1]  reversed
5  print(a[3:0:-1])   # [4, 3, 2]        stop 0 is excluded
6  print(a[3:1])      # []               step +1 cannot go down
```

4 A step of -1 with no start or stop walks the whole list backwards.

5 Counting down from index 3, stopping before index 0: indexes 3, 2, 1.

6 As with range, a step pointing away from stop gives an empty result, not an error.

Slice assignment and del

```
1  a = [1, 2, 3, 4, 5]
2  a[1:3] = [9]         # [1, 9, 4, 5]        two out, one in
3  a[1:1] = [7, 8]      # [1, 7, 8, 9, 4, 5]  empty slice: insert
4  del a[0]             # [7, 8, 9, 4, 5]
5  del a[1:3]           # [7, 4, 5]
6  print(a)
```

2 The replaced slice and the new values can have different lengths.

4-5 del removes one element, or a whole slice. del a[:] empties the list.

With a step, sizes must match: x[::2] = [1, 2] on 5 elements raises ValueError. del a removes the name a.

append, extend, insert, + and *

CODE, WITH A = [1, 2]	RESULT	CHANGES A?
`a.append(3)`	[1, 2, 3]	yes; returns None
`a.extend([3, 4])`	[1, 2, 3, 4]	yes; returns None
`a.insert(0, 9)`	[9, 1, 2]	yes; returns None
`a + [3]`	[1, 2, 3]	no: a new list
`a * 2`	[1, 2, 1, 2]	no: a new list

THE RULE

Every list method that changes the list returns None. The operators + and * build a new list instead.

One element, or each element

APPEND

Adds one element

```
a = [1, 2]
a.append([3, 4])
print(a, len(a))
```

[1, 2, [3, 4]] 3. The whole list goes in as a single element.

EXTEND

Adds each element

```
a = [1, 2]
a.extend([3, 4])
print(a, len(a))
```

[1, 2, 3, 4] 4. extend("hi") would add 'h' and 'i'.

ON THE EXAM

append always adds exactly one element, whatever you give it. extend adds as many as its argument holds.

insert(i, x) puts x in front of position i

```
1  x = [0, 1, 2]
2  x.insert(0, "a")      # ['a', 0, 1, 2]
3  x.insert(-1, "b")     # ['a', 0, 1, 'b', 2]   before the last element
4  x.insert(99, "c")     # ['a', 0, 1, 'b', 2, 'c']  past the end: appends
5  print(x)
```

3 insert(-1, x) does not add at the end. It goes in front of the current last element.

4 A position beyond the end is not an error. The item is simply appended.

pop returns what it removes. remove does not

```
1  a = [3, 1, 3, 2]
2  last = a.pop()          # 2      removes and returns the last
3  first = a.pop(0)        # 3      removes and returns index 0
4  a.remove(3)             #        removes the first 3 it finds
5  print(last, first, a)   # 2 3 [1]
6  print(a.remove(1))      # None
```

2-3 pop() takes the last element; pop(i) takes the one at index i. Both return it.

4 remove(x) searches by value, deletes the first match, and returns None.

clear() empties the list. del a[i] removes by position, like pop, but returns nothing.

When list methods fail

CODE	ERROR
`[].pop()`	IndexError: pop from empty list
`[1].remove(5)`	ValueError: list.remove(x): x not in list
`[1].index(5)`	ValueError. Python 3.14 message: list.index(x): x not in list
`[1, 2] + 3`	TypeError: can only concatenate list (not "int") to list

THE TIEBREAKER

Missing position: IndexError. Missing value: ValueError. Wrong type: TypeError.

index, count and in

```
1  a = [5, 3, 5, 1]
2  print(a.index(5))      # 0     the first position only
3  print(a.count(5))      # 2
4  print(3 in a, 9 not in a)   # True True
5  print(0 in [False])    # True: in compares with ==
```

2 index returns the position of the first match, even when there are several.

5 0 == False is True, so in finds it. Module four's rule again.

sort() and reverse() change the list and return None

```
1  a = [3, 1, 2]
2  a.sort()                # [1, 2, 3]
3  a.sort(reverse=True)    # [3, 2, 1]
4  a.reverse()             # [1, 2, 3]: just flips the current order
5  print(a.sort())         # None
```

3 reverse=True sorts from largest to smallest.

4 reverse() does not sort. It turns the list back to front, as it is now.

5 Printing the result of sort() prints None. The sorted list is in a itself.

What to carry into the exam

- Index 0 is first, -1 is last. Outside -n to n - 1: **IndexError**.
- Slices include start, exclude stop, and **never raise IndexError**.
- append adds one element; extend adds each. insert(-1, x) goes before the last.
- pop returns what it removes; remove, append, extend, insert, sort and reverse return None.
- Missing position: IndexError. Missing value: ValueError.

What is the output of this program?

```
1  data = [4, 0, 2, 1]
2  print(data[data[-1]] + data[data[1]])
```

A 4

B 1

C 5

D 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [4, 0, 2, 1]
2  print(data[data[-1]] + data[data[1]])
```

data[-1] is 1, so data[1] is 0. data[1] is 0, so data[0] is 4. 0 + 4 = 4.

A 4

Inner indexes first: data[1] is 0, data[0] is 4.

B 1

Adds the inner values, data[-1] + data[1]. Each is used as an index.

C 5

Reads data[-1] as 1 and adds data[0]. The first term is data[1].

D 2

Treats -1 as the second-last element. -1 is the last.

THE SIGNAL

For nested indexing, evaluate the innermost brackets first, then use the result as the index.

What is the output of this program?

```
1  a = ["p", "y", "t", "h", "o", "n"]
2  print(a[4:1:-1])
```

A `[]`

B `['h', 't', 'y']`

C `['o', 'h', 't', 'y']`

D `['o', 'h', 't']`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["p", "y", "t", "h", "o", "n"]
2  print(a[4:1:-1])
```

Indexes 4, 3, 2. Index 1 is the stop, so it is excluded.

A `[]`
The step is -1, so counting down from 4 to 1 is possible.

B `['h', 't', 'y']`
Starts at index 3. Start is included: index 4 is 'o'.

C `['o', 'h', 't', 'y']`
Includes index 1. Stop is excluded, even counting down.

D `['o', 'h', 't']`
Start 4 included, stop 1 excluded: indexes 4, 3, 2.

THE SIGNAL

With a negative step, start is still included and stop is still excluded.

What is the output of this program?

```
1  a = [1, 2]
2  a.append([3, 4])
3  a.extend([5, 6])
4  print(len(a))
```

A 6

B 5

C 4

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  a.append([3, 4])
3  a.extend([5, 6])
4  print(len(a))
```

a is [1, 2, [3, 4], 5, 6].

A 6

Treats append like extend. The appended list is one element.

B 5

2, plus 1 for the appended list, plus 2 from extend.

C 4

Treats extend like append.

D 3

Ignores the extend.

THE SIGNAL

append adds exactly one element. extend adds one per item of its argument.

What is the output of this program?

```
1  x = [1, 2, 3]
2  x.insert(-1, 9)
3  x.insert(0, x.pop())
4  print(x)
```

A `[3, 1, 2, 9]`

B `[9, 1, 2, 3]`

C `[1, 2, 3, 9]`

D `[3, 1, 2, 9, 3]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = [1, 2, 3]
2  x.insert(-1, 9)
3  x.insert(0, x.pop())
4  print(x)
```

After line 2: [1, 2, 9, 3]. pop() takes 3, leaving [1, 2, 9]. Insert 3 at the front.

A `[3, 1, 2, 9]`
9 goes before the last; pop moves the 3 to the front.

B `[9, 1, 2, 3]`
Assumes insert(-1) appends, so pop moves the 9.

C `[1, 2, 3, 9]`
Ignores line 3.

D `[3, 1, 2, 9, 3]`
pop removes the element it returns. The 3 is moved, not copied.

THE SIGNAL

insert(-1, x) lands before the last element. pop() removes and returns, so the element moves.

What is the output of this program?

```
1  a = [2, 4, 2, 6]
2  b = a.remove(2)
3  c = a.pop(1)
4  print(a, b, c)
```

A `[4, 6] 2 2`

B `[2, 6] None 4`

C `[4, 6] None 2`

D `[4, 2] None 6`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [2, 4, 2, 6]
2  b = a.remove(2)
3  c = a.pop(1)
4  print(a, b, c)
```

remove(2) deletes the first 2: [4, 2, 6]. pop(1) takes index 1, which is 2: [4, 6].

A `[4, 6] 2 2`
remove returns None, not the value removed.

B `[2, 6] None 4`
Treats remove(2) as removing index 2. It removes the value 2.

C `[4, 6] None 2`
remove deletes the first 2 and returns None; pop(1) returns the other 2.

D `[4, 2] None 6`
Treats pop(1) as popping the last element.

THE SIGNAL

remove works by value and returns None. pop works by position and returns the element.

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:4] = [0]
3  del a[-1]
4  print(a)
```

A [1, 0, 4, 5]

B [1, 0]

C [1, 0, 0, 0]

D [0, 5]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:4] = [0]
3  del a[-1]
4  print(a)
```

a[1:4] is [2, 3, 4]. Replaced by [0]: [1, 0, 5]. Then del a[-1]: [1, 0].

A `[1, 0, 4, 5]`

Replaces only indexes 1 and 2. Stop 4 means indexes 1, 2 and 3.

B `[1, 0]`

Three elements replaced by one, then the last one deleted.

C `[1, 0, 0, 0]`

Assumes the 0 fills every slot. The slice shrinks to one element.

D `[0, 5]`

Starts the slice at index 0. Index 1 is the 2.

THE SIGNAL

Slice assignment without a step can change the list's length.

> my_list = [5, 4, 3, 2] has been assigned. Each expression is evaluated on its own.

Which two expressions raise an exception? (Choose two.)

A `my_list[-4]`

B `my_list[4]`

C `my_list[1:10]`

D `my_list.index(1)`

◆ PAUSE AND DECIDE

Which two expressions raise an exception? (Choose two.)

A `my_list[-4]`

-4 is the first element of four. 5.

B `my_list[4]`

Valid indexes stop at 3. IndexError.

C `my_list[1:10]`

Slices never raise IndexError. [4, 3, 2].

D `my_list.index(1)`

1 is not in the list. ValueError.

THE SIGNAL

Indexing checks bounds; slicing does not. index() of a missing value is a ValueError.

What is the output of this program?

```
1  data = [0, None, [], "", [[]]]
2  print(len(data), [] in data, 0 in data[4])
```

A 4 True False

B 5 False False

C 5 True False

D 5 True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [0, None, [], "", [[]]]
2  print(len(data), [] in data, 0 in data[4])
```

data[4] is [[]]: a list holding one empty list. 0 == [] is False.

A `4 True False`
Every element counts, even None and empty ones.

B `5 False False`
data[2] is an empty list, and [] == [] is True.

C `5 True False`
Five elements; [] is data[2]; 0 does not equal [].

D `5 True True`
0 is not in [[]]. Its only element is [], and 0 == [] is False.

THE SIGNAL

len counts every top-level element, empty or not. in compares each element with ==.